# Deepfake Image Detection
# Everything You Need to Know

A complete walkthrough of the project, written so that a reader with no machine-learning background can follow it, plus a full set of interview questions and model answers.

# 1. The big picture

## 1.1 What this project does

The project is a **binary image classifier**: a program that looks at a photograph of a human face and outputs a probability that the face is **real** (an unedited photograph) or **fake** (digitally manipulated). It is built with TensorFlow/Keras in Python, using a technique called **transfer learning** on top of a pre-trained network called **MobileNetV2**.

It is not only a model. It is a complete, engineered system: a reproducible data split, a configurable training pipeline, a rigorous evaluation protocol, an explainability tool (Grad-CAM), an inference contract for deployment, and a suite of 57 automated tests. The same method exists in two forms: a self-contained Jupyter notebook for reading and demonstration, and a tested Python package with a command-line interface for experiments.

## 1.2 Why the problem matters

Manipulated face images are used for fraud, misinformation and harassment. Automatic detection is an active research field (image forensics). This project tackles a specific, well-defined slice of it: distinguishing expert Photoshop face composites from genuine photographs, using a public benchmark dataset.

## 1.3 The 30-second pitch (memorise this)

*"I built a deepfake face detector using MobileNetV2 transfer learning on the CIPLAB real-and-fake-faces dataset, about 2,000 images. The interesting part is the engineering discipline: a sealed test set that no training decision ever touches, preprocessing and augmentation inside the model so training and serving can never disagree, every metric reported against the majority-class baseline with bootstrap confidence intervals, probability calibration with an abstain band for uncertain cases, and 57 automated tests. It reaches about 0.70 ROC AUC, which I report honestly as modest, and I can explain exactly why the architecture limits it and what I would do next."*

# 2. The dataset

The project uses the **CIPLAB real-and-fake-face-detection** dataset published on Kaggle by Yonsei University's Computational Intelligence and Photography Lab.

| Fact | Value |
|---|---|
| Total images | 2,041 colour face photographs (roughly 600 x 600 pixels) |
| Real faces | 1,081 (class label 1) |
| Fake faces | 960 (class label 0) |
| How the fakes were made | Expert Photoshop composites: real photos with eyes, nose or mouth swapped in from other photos |
| What the fakes are NOT | GAN or diffusion output. The artefacts are blending seams and lighting mismatches, not generator fi |
| Split used | 70% train (1,429) / 15% validation (306) / 15% test (306) |

## 2.1 Metadata hidden in the filenames

Fake images are named like `easy_100_1111.jpg`. The parts mean:

- **easy / mid / hard**: how difficult the fake is for a *human* to spot.
- **The 4-bit mask (e.g. 1111)**: which regions were manipulated, in order: left eye, right eye, nose, mouth. `1111` means all four were replaced; `0010` means only the nose.

The data pipeline parses this metadata into the split manifest, which later allows accuracy to be reported per difficulty tier.

## 2.2 Why the dataset choice matters

Because the fakes are Photoshop composites, a model trained here learns to spot **blending and lighting inconsistencies**. It should not be expected to transfer to GAN-generated faces (like thispersondoesnotexist.com), which carry different artefacts. This is a known property of forgery detectors: they generalise poorly across manipulation families, which is why cross-dataset evaluation is listed as future work.

# 3. Key concepts in plain English

Every technical term the project relies on, explained without jargon. If you can explain each of these in your own words, you can survive any drill-down question.

### Neural network / CNN

A function with millions of adjustable numbers (weights) that learns patterns from examples. A Convolutional Neural Network (CNN) is the variant specialised for images: it slides small filters across the picture to detect edges, textures and shapes, building up from simple to complex features layer by layer.

### ImageNet

A famous dataset of 1.2 million labelled photos across 1,000 everyday categories. Networks pre-trained on it have already learned general visual features (edges, textures, object parts) that transfer to other image tasks.

### Transfer learning

Instead of training a network from scratch (which needs huge data), take one already trained on ImageNet, keep its learned feature extractor, and attach a small new 'head' trained on your task. Essential here because 2,041 images is far too few to train a CNN from scratch.

### MobileNetV2

A small, efficient CNN architecture (about 2.3 million parameters) designed to run on phones. Chosen because the dataset is small (a small model overfits less) and training happens on a CPU.

### Backbone vs head

The backbone is the pre-trained MobileNetV2 feature extractor. The head is the small new part added on top: global average pooling, dropout, and one output neuron. Stage 1 trains only the head; stage 2 gently fine-tunes the top of the backbone too.

### Train / validation / test split

Three disjoint sets of images. **Train**: the model learns from these. **Validation**: used to make decisions (when to stop, which checkpoint to keep, what threshold to use). **Test**: sealed away and used exactly once at the end, to produce the honest final numbers. If a decision is made using test data, the reported result becomes optimistically biased.

### Stratified split

The split preserves the real/fake ratio inside every subset, so no subset is accidentally dominated by one class.

### Epoch, batch, learning rate

An **epoch** is one full pass over the training data. A **batch** is the group of images (32 here) processed before one weight update. The **learning rate** is the step size of each update: too large destroys pre-trained knowledge, too small learns nothing.

### Overfitting

When a model memorises the training images instead of learning general patterns, so training accuracy is high but performance on new images is poor. Countered here by a small frozen backbone, dropout, data augmentation and early stopping.

### Data augmentation

Randomly altering training images (small flips, rotations, zooms, contrast shifts) every epoch, so the model sees fresh variations and cannot memorise exact pixels. Deliberately mild here: aggressive photometric noise would erase the very forensic evidence (blending seams, noise patterns) the model needs.

### BatchNormalization (BN)

Layers inside the backbone that normalise activations using running statistics estimated over ImageNet's million images. They are kept frozen during fine-tuning: letting 1,400 images overwrite statistics estimated from a million would destroy the pre-trained features.

### Logit vs probability

The model's last layer outputs a raw unbounded score called a **logit**. Passing it through the **sigmoid** function squashes it to a probability between 0 and 1. Here sigmoid(logit) = P(real). Training on logits (from_logits=True) is numerically stable, and Grad-CAM works better on logits.

### Loss function

The quantity training minimises. Binary cross-entropy measures how far predicted probabilities are from the true 0/1 labels, punishing confident mistakes heavily.

### Early stopping and checkpointing

Watch a validation metric during training; stop when it stops improving, and keep the weights of the best epoch (not the last one). The direction (maximise or minimise) is stated explicitly in this project because Keras' automatic inference gets AUC wrong.

### Threshold / operating point

The probability cut-off that turns a score into a decision. 0.5 is only correct for balanced classes and symmetric costs. Here the threshold is chosen on the validation split using Youden's J statistic and then shipped as part of the model's deployment contract.

### Accuracy, precision, recall, F1

**Accuracy**: fraction of correct decisions. **Precision** (for class X): of everything predicted X, how much really was X. **Recall**: of everything that really was X, how much was found. **F1**: harmonic mean of precision and recall.

### Majority-class baseline

The accuracy you get by always predicting the most common class. Here 52.94% (always say 'real'). Any reported accuracy only means something relative to this floor; the difference is called **lift**.

### Confusion matrix

A 2x2 table counting the four outcomes: fakes called fake, fakes called real, reals called fake, reals called real. Shows where the errors actually are.

### ROC AUC

Area under the ROC curve. The probability that a randomly chosen fake gets a lower P(real) score than a randomly chosen real image. 0.5 is coin-flipping, 1.0 is perfect. It is **threshold-free**, which makes it the most trustworthy headline figure: accuracy changes when only the cut-off moves, AUC does not.

### PR AUC

Area under the precision-recall curve, computed per class. More informative than ROC AUC when classes are imbalanced.

### Calibration, Brier score, log loss, ECE

A model is **calibrated** if, among images given 70% confidence, about 70% are actually correct. The **Brier score** is the mean squared error between predicted probability and truth (lower is better). **Log loss** punishes confident errors sharply. **Expected Calibration Error (ECE)** summarises the gap between stated confidence and actual accuracy.

### Bootstrap confidence interval

Resample the test set with replacement 2,000 times, recompute the metric each time, and take the middle 95% of results. On a 306-image test set the interval for accuracy is about plus or minus 5 points, which is why a single accuracy number without an interval is nearly meaningless.

### Grad-CAM

An explainability technique: take the gradient of the output with respect to the last convolutional feature map, and use it to produce a heatmap of which image regions most influenced the decision. Caveat: it shows where the model looked, not whether it was right.

### Abstain band

A margin around the threshold in which the system refuses to decide and routes the image to a human reviewer. At this accuracy level, a forced binary verdict about a real person would be irresponsible; abstention is a design feature.

### tf.data pipeline

TensorFlow's input machinery: read files, decode JPEGs, resize, cache, shuffle, batch, prefetch. Order matters; details in section 5.3.

### Ablation

An experiment that changes exactly one variable (e.g. input resolution) against the baseline, so any difference in results can be attributed to that one change.

# 4. How the project is organised

## 4.1 Two independent interfaces

| | Deepfake Detection.ipynb | src/deepfake/ + CLI |
|---|---|---|
| Purpose | Reading, demoing, one reproducible run | Ablations, tests, automation |
| Needs | open.bat and a browser | PYTHONPATH=src and a terminal |
| Config | The CONFIG cell inside it | config.yaml |
| Output | artifacts/notebook/ | artifacts/<run_name>/ |
| Tests | None (it is the demo) | 57 tests |

They are **independent implementations of the same method**; neither imports the other. The notebook can never break because of a refactor in the package. The cost is that a change to the method must be made in both places; the benefit is that the notebook is genuinely standalone and ships already executed, with all outputs and figures visible.

## 4.2 File layout

```
config.yaml              every tunable value; nothing hardcoded in src/
src/deepfake/
  config.py              load + validate config, resolve paths
  data.py                discovery, stratified split manifest, tf.data pipeline
  model.py               backbone + head; preprocessing/augmentation as LAYERS
  train.py               two-stage training, explicit callback directions
  evaluate.py            sealed-test metrics, bootstrap CIs, calibration
  predict.py             inference against the saved deployment contract
  gradcam.py             attention maps taken from the logit
  cli.py                 one entry point for all of the above
tests/                   57 tests; each pins a behavioural guarantee
experiments/
  derive.py              generate a config variant by dotted override
  compare.py             table of every finished run, from metrics.json
Deepfake Detection.ipynb the standalone notebook; all logic inline
artifacts/<run_name>/    model.keras, label_map.json, metrics.json, figures
data/                    the raw dataset (not tracked)
```

## 4.3 One rule governs everything

**config.yaml is the only place a number lives.** No module hardcodes a learning rate, path, threshold or split fraction. Every run copies the exact configuration it used into its output folder as `config.used.yaml`, so any result can be traced back to the exact settings that produced it. Running an experiment means deriving a one-line config variant, never editing code.

## 4.4 The CLI commands

| Command | Does |
|---|---|
| python -m deepfake.cli split | Builds the stratified train/val/test manifest CSV |
| python -m deepfake.cli train | Two-stage transfer learning, saves model.keras |
| python -m deepfake.cli evaluate | Sealed-test evaluation, writes metrics.json |
| python -m deepfake.cli gradcam | Attention figure for sample test images |
| python -m deepfake.cli predict a.jpg | Classifies image files |

| Command | Does |
| --- | --- |
| python -m deepfake.cli smoke | 30-second end-to-end wiring check |
| python -m deepfake.cli all | split, train, evaluate, gradcam in sequence |
| pytest | The 57-test suite; no GPU or dataset needed |

# 5. The pipeline, step by step

This mirrors the order of the notebook and of `cli all`. Each step says what happens and why it is done that way.

## Step 1: Discovery and the split manifest

The code walks the two dataset folders once and writes a CSV manifest with one row per image: path, class, label (0=fake, 1=real), difficulty tier, region mask, and which split it belongs to. The split is stratified per class and seeded.

**Why a materialised CSV?** Every later command (train, evaluate, Grad-CAM) reads the same file, so they can never disagree about which image is in which split. The split is auditable, reproducible, and disjointness is asserted on creation.

## Step 2: The three-way split with a sealed test set

70% train / 15% validation / 15% test. Validation drives *every* decision: early stopping, checkpoint selection, threshold choice. The test set is opened by exactly one module, evaluate.py, exactly once, after all decisions are frozen. This is what makes the final numbers an unbiased estimate of real-world performance.

## Step 3: The input pipeline

```
from_tensor_slices(paths, labels)
  -> map(read + decode + resize)   expensive, deterministic
  -> cache()                       decode happens once
  -> shuffle()                     train only; val/test keep file order
  -> batch(32)
  -> prefetch()
```

The pipeline emits **raw float32 pixels in [0, 255]**. It never normalises and never augments; both of those live inside the model as layers. That ordering makes the cache safe (nothing random is cached) and makes the serving path identical to the training path.

## Step 4: The model

```
Input (224, 224, 3), raw pixels in [0, 255]
  -> Augmentation layer    RandomFlip(horizontal), mild rotate/zoom/
                           translate/contrast    (active in training only)
  -> Rescaling(1/127.5, offset=-1)      maps to [-1, 1] for MobileNetV2
  -> MobileNetV2 backbone  ImageNet weights, include_top=False,
                           called with training=False (BN pinned to
                           inference mode for the model's whole life)
  -> GlobalAveragePooling2D
  -> Dropout(0.2)
  -> Dense(1, activation=None)          a LOGIT, not a probability
```

Three load-bearing choices:

- **Preprocessing is a layer.** One implementation, carried inside the saved model, so training and serving arithmetic can never drift apart.
- **Augmentation is a layer.** It re-randomises every batch of every epoch and switches itself off at inference. Placed in a pipeline before cache(), the random transforms would be frozen after epoch 1 and replayed forever.
- **The output is a linear logit.** Training uses BinaryCrossentropy(from_logits=True) for numerical stability, and Grad-CAM differentiates the logit instead of a saturated sigmoid whose gradients vanish.

## Step 5: Two-stage training

| Stage | Backbone | Learning rate | Purpose |
|---|---|---|---|
| 1 | Frozen | 1e-4 | Fit the new head without letting large random gradients touch pre- |
| 2 | Top 40 layers unfrozen, all BN still frozen | 1e-5 (10x lower) | Gently adapt the most task-specific features without destroying th |

- **Callback directions are explicit.** EarlyStopping and ModelCheckpoint both monitor val_auc_roc with mode='max' written out. Keras' default mode='auto' infers direction from the metric *name* and would resolve 'auc' to minimise, silently saving the worst epoch. The config loader refuses files that omit the mode.
- **Exactly one learning-rate controller.** ReduceLROnPlateau alone; running a scheduler alongside it would overwrite every reduction at the next epoch.
- **Each stage checkpoints to its own file.** Stage 2's callback starts with a fresh 'best so far', so a shared file could be silently overwritten by a worse stage-2 epoch. The genuinely better stage is copied to model.keras.
- **Recompiling between stages is mandatory.** A trainability change only takes effect when the training function is rebuilt.

## Step 6: Evaluation protocol

- Predict on **validation**; choose the operating threshold there (Youden's J: maximise true-positive rate minus false-positive rate).
- Predict on **test**; apply that threshold unchanged; compute everything.
- Write the threshold into label_map.json: the operating point is part of the deployment contract, not an evaluation detail.

Reported: accuracy with the majority baseline and a bootstrap 95% CI, balanced accuracy, ROC AUC, PR AUC per class, log loss, Brier score, a calibration curve with ECE, the confusion matrix, per-class precision/recall/F1, accuracy per difficulty tier, abstain-band behaviour, and worst_mistakes.csv (the most confident errors, for failure analysis).

## Step 7: Grad-CAM explainability

Rather than looking a layer up by name (which fails because MobileNetV2 is nested inside the outer model), the implementation takes the tensor feeding GlobalAveragePooling2D, which *is* the backbone's final feature map, and differentiates the logit. Works with any backbone; no layer name is mentioned.

## Step 8: Inference and the deployment contract

The Predictor loads **model.keras and label_map.json together**. The JSON carries the class order, image size, colour order, preprocessing description, the statement that the output is a logit, and the operating threshold. The predicted name is always `class_names[int(p >= threshold)]`; no class name exists as a string literal in the decision path, so polarity cannot silently flip. Predictions inside the abstain margin return 'abstain' instead of a class.

## Step 9: The test suite

57 tests run without a GPU or the dataset: a fixture generates a synthetic 26-image dataset in a temp directory. Highlights: splits are disjoint and reproducible; val/test are never shuffled; preprocessing maps [0,255] to [-1,1]; the output is an unsquashed logit; unfreezing keeps BatchNorm frozen; class and label

can never disagree; the majority baseline is always reported; and an AST-based test bans np.random.uniform and r2_score from every module, guaranteeing that every reported metric is a measured classification statistic.

# 6. The results and what they mean

## 6.1 Headline numbers (sealed test split, 306 images, CLI baseline run)

| Metric | Value | How to read it |
|---|---|---|
| Accuracy | 0.6405 (95% CI 0.588 - 0.693) | Majority baseline is 0.5294, so the lift is +0.111 |
| Balanced accuracy | 0.6424 | Both classes contribute equally |
| ROC AUC | 0.6969 (95% CI 0.639 - 0.753) | Chance is 0.5; the most trustworthy single figure |
| PR AUC (fake / real) | 0.6827 / 0.7017 | Ranking quality per class |
| Log loss | 0.6461 | Probability quality |
| Brier score | 0.2242 | Mean squared error of the probabilities |
| Calibration error (ECE) | 0.0662 | Mildly over-confident |
| Operating threshold | 0.6116 | Chosen on validation by Youden's J |

## 6.2 Confusion matrix (rows = actual, columns = predicted)

| | Predicted fake | Predicted real | Recall |
|---|---|---|---|
| Actual fake (144) | 97 | 47 | 0.674 |
| Actual real (162) | 63 | 99 | 0.611 |

## 6.3 Accuracy by human difficulty tier of the fakes

| Tier | n | Accuracy |
|---|---|---|
| easy | 32 | 0.469 |
| mid | 81 | 0.741 |
| hard | 31 | 0.710 |
| none (real images) | 162 | 0.611 |

Counter-intuitively the model is **worst on the 'easy' tier**. 'Easy' means easy for a human, which is not the same as easy for a CNN; tier sample sizes are also small (about 32), so the differences are not strongly significant. It is exactly the kind of insight a single accuracy number hides.

## 6.4 The abstain band

With a margin of 0.10 around the threshold, 98 of 306 test images (32%) are routed to human review and accuracy on the remaining 208 rises to **0.697**. That is the responsible deployment story: the system decides only when reasonably confident.

## 6.5 The ablations: four variants, none beat the baseline

| Run | Accuracy | 95% CI | ROC AUC | Brier |
|---|---|---|---|---|
| unfreeze_all (154 layers) | 0.6176 | [0.565, 0.670] | 0.6974 | 0.2270 |
| baseline (224 px, top 40) | 0.6405 | [0.588, 0.693] | 0.6969 | 0.2242 |
| res384 (384 px input) | 0.5817 | [0.526, 0.637] | 0.6864 | 0.2276 |

| Run | Accuracy | 95% CI | ROC AUC | Brier |
|---|---|---|---|---|
| srm (frequency branch) | 0.6373 | [0.585, 0.690] | 0.6743 | 0.2334 |

**Every confidence interval overlaps every other.** No difference is established. The baseline is kept because it is the simplest and cheapest, not because it 'won'. Two humbling findings: higher resolution had been predicted to be the biggest win and came last while costing 3x the training time; and the SRM frequency branch, the most domain-specific idea, scored lowest on AUC (a second stream trained from scratch is too many new parameters for 1,429 images). The deeper lesson: when four changes in a row land inside the noise band, the *evaluation* is the bottleneck, and the next investment should be k-fold cross-validation, not another architecture tweak.

## 6.6 Why the accuracy is modest, in one paragraph

Forgery cues are **high-frequency and local**: blending seams, noise inconsistencies, compression history. This pipeline destroys them twice. Resizing 600 px images down to 224 throws away fine detail, and global average pooling collapses the final 7x7 spatial grid into 1,280 numbers that no longer say *where* anything happened. A frozen ImageNet backbone was also never trained to look for such statistics. The limit is architectural, not a bug, and the future work list (face cropping at native resolution, frequency-domain features, region-mask supervision) targets exactly this.

# 7. Design decisions cheat sheet

The project keeps an append-only decision log (DECISIONS.md). One line each:

| ID | Decision in one line |
|---|---|
| D-001 | All tunables live in one validated config.yaml; nothing hardcoded |
| D-002 | The split is materialised to a CSV manifest, never recomputed |
| D-003 | Three-way split; the test set is sealed until evaluation |
| D-004 | Preprocessing (rescaling to [-1,1]) is a layer inside the model |
| D-005 | Augmentation is a model layer, not a pipeline stage |
| D-006 | Augmentation is conservative and forensics-aware (no vertical flips, minimal photometric jitter) |
| D-007 | BatchNormalization stays frozen and in inference mode, always |
| D-008 | The output is a linear logit; the loss uses from_logits=True |
| D-009 | Callback directions (mode) are required config keys, never inferred |
| D-010 | Exactly one learning-rate controller runs |
| D-011 | Each training stage checkpoints to its own file; the genuine winner ships |
| D-012 | The predicted class name is derived from the index, never a string literal |
| D-013 | Classification metrics only; no R-squared; AST-enforced by a test |
| D-014 | Every headline number ships with a baseline and a bootstrap CI |
| D-015 | The threshold is chosen on validation and shipped in the contract |
| D-016 | An abstain band routes uncertain cases to human review |
| D-017 | Models are saved with model.save() to .keras, never pickled |
| D-018 | A tested package with a CLI is the experiment surface |
| D-019 | Redundant dataset copies are reported, not auto-deleted |
| D-020 | Default threshold strategy is Youden's J (per-class F1 degenerated) |
| D-021 | The baseline config is kept because four measured alternatives failed to beat it |
| D-022 | The standalone notebook is the primary reading interface; the package is the experiment interface |

# 8. Limitations and future work

## 8.1 Stated limitations

- **Modest absolute accuracy.** AUC about 0.70 is well above chance but not deployable as an automatic decision-maker. Architectural cause explained in 6.6.
- **Identity leakage risk.** The split is stratified by class, not by source identity; CIPLAB fakes are built from real photos in the same collection, so some identity overlap across splits is possible and numbers may be slightly optimistic.
- **Single dataset, single manipulation family.** Photoshop composites only; no cross-dataset evaluation (FaceForensics++, Celeb-DF) has been run yet.
- **Not fully deterministic.** Seeds fix the split, initialisation and shuffle, but multi-threaded tf.data and CPU kernels still move the last digits.

- **Ethical boundary.** The project states plainly: at this accuracy the output must never be used as evidence about a real person; a positive means 'worth a human look', nothing more.

## 8.2 Future work, in the order expected to pay off

- **K-fold cross-validation** first: the ablations proved a single 306-image test split cannot resolve few-point differences, so better evaluation now beats any model tweak.
- **Face cropping** (MTCNN/RetinaFace/MediaPipe) and classifying the aligned crop at native resolution, instead of downsampling the whole frame.
- **Region-mask auxiliary supervision**: the manifest already parses which facial regions were manipulated; predicting them as a multi-label side task is cheap regularisation and makes Grad-CAM measurable.
- **Random JPEG re-compression augmentation**: the forensically correct noise to be robust to.
- **Cross-dataset evaluation**: expect a large drop and report it; that is the finding, not a failure.

# 9. Interview questions and answers

## A. Project overview and warm-up

### Q1. Walk me through your project in two minutes.

Start with the pitch from section 1.3, then expand: the dataset and its Photoshop nature, the two interfaces, the pipeline steps (split, model, two-stage training, sealed evaluation), the headline numbers with the baseline, and close on the abstain band and future work. Practise saying it aloud twice.

### Q2. What problem does this solve and who would use it?

It screens face photographs for manipulation. Realistic users: content-moderation teams, journalism fact-checkers, KYC/identity-verification pipelines. In all cases as a triage tool that flags images for human review, not as a final judge.

### Q3. Why did you choose this project?

It combines a socially relevant problem with deep engineering content: transfer learning, evaluation methodology, calibration, explainability and testing. It also gave room to demonstrate scientific honesty: reporting a modest result correctly is harder and more valuable than inflating it.

### Q4. What are you most proud of in it?

The measurement discipline. A sealed test set, baseline and confidence interval on every number, a threshold chosen only on validation, an AST-level test that makes fabricating a metric impossible, and four ablations reported honestly as ties.

### Q5. What was the hardest part?

Getting the evaluation right. Anyone can call model.fit; making sure the reported number is an unbiased estimate (no test leakage, explicit callback directions, threshold on validation only) is where most projects silently go wrong.

## B. Data and preprocessing

### Q6. Describe the dataset. Is it balanced?

CIPLAB real-and-fake faces: 2,041 images, 1,081 real and 960 fake, roughly 53/47, near-balanced. Fakes are expert Photoshop composites with eyes, nose or mouth swapped; filenames encode a human difficulty tier and a 4-bit mask of manipulated regions, which the manifest parses.

### Q7. How did you split the data and why that way?

70/15/15 stratified by class, seeded, written once to a CSV manifest that every command reads. Validation drives all decisions; the test set is read exactly once by evaluate.py after every decision is frozen. Stratification preserves class ratios; the manifest guarantees all components agree on the split.

### Q8. Why do you need a test set separate from validation?

Because the validation set stops being neutral the moment it drives decisions: early stopping, checkpoint selection and threshold choice all tune towards it. Numbers reported on it are optimistically biased by construction. Only data no decision ever touched gives an honest generalisation estimate.

### Q9. Why is your augmentation so mild?

Two reasons. Faces are never upside down at inference, so vertical flips teach a useless invariance. More importantly, heavy photometric jitter destroys exactly the low-level statistics (noise consistency, blending seams, compression history) that distinguish a composite from an original. Generic augmentation recipes actively hurt forensics tasks.

### Q10. Where does normalisation happen and why there?

Inside the model, as a Rescaling(1/127.5, offset=-1) layer mapping [0,255] to [-1,1], which MobileNetV2's ImageNet weights expect. Keeping it in the model means one implementation carried by the saved file, so training and serving can never use different arithmetic.

### Q11. What is the risk in the order of tf.data operations?

cache() placement. If it comes after an augmentation map, the first epoch's random transforms are stored and replayed forever, silently freezing augmentation. Here cache sits after decode/resize (deterministic work) and augmentation lives in the model, so the trap cannot occur.

### Q12. Is there data leakage in your project?

One acknowledged risk: the split is stratified by class but not by source identity, and CIPLAB fakes are composited from real photos in the same collection, so the same face can appear across splits. Reported numbers may be slightly optimistic. The fix would be an identity-disjoint split, listed as future work.

## C. Model and training

### Q13. Why MobileNetV2 and not a bigger model like ResNet or EfficientNet?

With 1,429 training images, capacity is a liability: a bigger backbone overfits faster and trains slower on CPU. MobileNetV2 is small (about 2.3M parameters), well-studied, and the config supports EfficientNetB0 as a drop-in if warranted. The measured bottleneck turned out to be information reaching the head, not backbone capacity.

### Q14. Explain transfer learning and your two-stage schedule.

The backbone arrives with ImageNet features. Stage 1 freezes it entirely and trains only the new head at learning rate 1e-4, so large random gradients from the fresh head cannot wreck pre-trained weights. Stage 2 unfreezes the top 40 layers at 1e-5, ten times lower, nudging the most task-specific features. Each stage checkpoints separately and the genuinely better one ships.

### Q15. Why do you keep BatchNormalization frozen even while fine-tuning?

BN layers carry moving mean/variance statistics estimated over a million ImageNet images. Unfreezing them lets a 1,400-image dataset overwrite those statistics, which destabilises every layer above. The backbone is also called with training=False so BN always runs in inference mode.

### Q16. Why does the model output a logit instead of a probability?

Numerical stability: BinaryCrossentropy(from_logits=True) fuses the sigmoid into the loss and avoids log(0). And explainability: Grad-CAM differentiates the logit; a saturated sigmoid has vanishing gradients, producing noise heatmaps exactly on confident predictions. Inference applies the sigmoid explicitly, and label_map.json records output: logit so nothing can double-apply it.

### Q17. What is the mode='auto' trap you guard against?

Keras infers a callback's direction from the metric name: names containing 'acc' are maximised, everything else minimised. 'val_auc' contains 'auc', not 'acc', so the default silently minimises AUC: the checkpoint saves the worst epoch and restore_best_weights restores it. Here mode is a required config key; loading fails without it, and a test pins that behaviour.

### Q18. Why exactly one learning-rate controller?

LearningRateScheduler sets the rate every epoch; ReduceLROnPlateau multiplies the current rate. Together, the scheduler overwrites every reduction at the next epoch, so the trajectory becomes an artefact of callback ordering. The config selects one.

### Q19. Why must you recompile between stage 1 and stage 2?

Changing layer.trainable only takes effect when the training function is rebuilt; without recompiling, the 'unfrozen' layers would still not receive gradients. initial_epoch carries the epoch counter forward so history stays continuous.

### Q20. How do you handle class imbalance?

The dataset is near-balanced (53/47), so heavy machinery is unnecessary; the pipeline computes class weights only when imbalance exceeds a configured ratio. The evaluation side matters more: balanced accuracy, per-class PR AUC, and a cost-neutral threshold via Youden's J.

### Q21. What would happen if you trained the whole backbone from scratch?

Severe overfitting: 2.3M parameters against 1,429 images. The measured ablation that unfroze all 154 layers (still with pre-trained initialisation) already showed no improvement; random initialisation would be strictly worse.

### Q22. How long does training take and on what hardware?

About 35 epochs across both stages, roughly 5 minutes on a plain CPU; the notebook re-runs end to end in about 8 minutes. No GPU is required anywhere, including the test suite.

## D. Evaluation and metrics

### Q23. Which single metric do you trust most and why?

ROC AUC (0.6969), because it is threshold-free. Accuracy moves when only the operating point changes; the ablations proved it: switching threshold strategy moved accuracy from 0.536 to 0.6405 while AUC stayed 0.6969, on the same model.

### Q24. Your accuracy is 64%. Is that good?

Framed correctly: the majority baseline is 52.9%, so the lift is +11.1 points, and the 95% bootstrap CI is [0.588, 0.693]. It is a real signal, clearly above chance, and honestly modest. I can explain the architectural ceiling and what would raise it. Practitioners on this dataset with similar setups report comparable numbers.

### Q25. Why report a majority-class baseline at all?

Because accuracy without a floor is uninterpretable. 'Predict real for everything' already scores 52.9%; only the distance above that floor is model skill.

### Q26. How do you compute the confidence intervals?

Percentile bootstrap: resample the 306 test predictions with replacement 2,000 times, recompute the metric each time, take the 2.5th and 97.5th percentiles. On this sample size the accuracy interval spans about 10 points, which changes how results should be described.

### Q27. Explain how you chose the decision threshold.

On the validation split only, by Youden's J: the threshold maximising TPR minus FPR. It is then applied unchanged to test and shipped inside label_map.json. The original default (maximise F1 for the fake class) degenerated: F1 stays high while the classifier drifts towards predicting one class for everything, which produced fake recall 0.93 against real recall 0.19. Measured, documented, superseded.

### Q28. What is calibration and how calibrated is your model?

Calibration asks whether stated confidence matches empirical accuracy. Measured with a reliability curve, ECE 0.066 (mildly over-confident), Brier 0.224 and log loss 0.646. It matters because downstream users treat the probability as meaning something; an over-confident forensics model is dangerous.

### Q29. Why is there no R-squared in the project?

R-squared measures explained variance of a continuous target; for a Bernoulli 0/1 label it is not interpretable. The correct probabilistic scores are Brier, log loss and calibration error. An AST-parsing test additionally bans r2_score and np.random.uniform from every module, so no metric can ever be produced by anything but measurement. If a stakeholder insists on an R-squared-shaped number, McFadden's pseudo R-squared is the defensible analogue.

### Q30. What does the confusion matrix tell you?

Fake recall 0.674 versus real recall 0.611: it catches two thirds of fakes while wrongly flagging about 39% of real images. For a triage system, the false-positive rate on real images is the cost driver, which is exactly what the abstain band manages.

### Q31. Explain the abstain band.

A margin of 0.10 around the threshold in which the system returns 'abstain' instead of a class. On test, 32% of images abstain and accuracy on the remainder rises to 0.697. It converts a weak binary decider into a useful triage tool and the trade-off is measured, not assumed.

### Q32. Your four ablations all failed. What did you learn?

Three things. Higher resolution, the change I predicted would help most, came last while tripling training time: a frozen, lightly fine-tuned backbone cannot exploit extra detail. The SRM frequency branch failed because a stream trained from scratch is too many parameters for 1,429 images. And most importantly: when four changes land inside overlapping confidence intervals, the evaluation itself is the bottleneck, so the next step is k-fold cross-validation, not more architecture.

### Q33. How would you know if your model is just memorising identities?

Build an identity-disjoint split (cluster by source face, keep clusters within one split) and compare. A large drop would confirm identity leakage. This is a known limitation I state up front.

## E. Engineering, testing and reproducibility

### Q34. How do you make experiments reproducible?

Single source of truth: config.yaml holds every tunable; the loader validates it; each run copies config.used.yaml into its artifact folder; the split manifest is materialised and seeded; experiments derive one-line config variants via experiments/derive.py so the diff against baseline is exact.

### Q35. What do your 57 tests actually test, with no GPU and no dataset?

A fixture writes a synthetic 26-image dataset to a temp directory. Tests pin behavioural guarantees: split disjointness and reproducibility, no shuffling of val/test, correct [-1,1] preprocessing, logit output, BN staying frozen after unfreeze, class/label agreement across logits, baseline always reported, CI bracketing, threshold and abstain honoured, CLI argument handling, and the AST ban on random or regression metrics.

### Q36. Why save to .keras instead of pickle/joblib?

Pickling a live Keras object ties the artifact to one exact TensorFlow build, Python version and pickle protocol, and unpickling untrusted files is a security risk. model.save() to the Keras v3 format is supported, survives upgrades, and converts to SavedModel, ONNX or TFLite.

### Q37. What is in your deployment contract and why?

label_map.json ships with the model: class order, image size, colour order, preprocessing description, 'output: logit', and the operating threshold. Anything an integrator would otherwise guess is written down; the Predictor refuses to run if the contract disagrees with expectations.

### Q38. Why do the notebook and the package not share code?

Deliberate duplication. The notebook's whole value is being standalone and readable top to bottom; importing from src/ would make it unreadable alone and breakable by refactors. The cost (changes made twice) is bounded because the notebook is a fixed demonstration while experiments run through the package.

### Q39. How would you turn this into an API service?

Wrap Predictor in a FastAPI endpoint: accept an image upload, validate size/type, run predict, return class, probability, abstain flag and model version from the contract. Batch requests for throughput, add logging of inputs and scores for monitoring drift, keep the model file and threshold versioned together.

## F. Scenario, ethics and forward-looking

### Q40. If you had one more week, what would you do first?

Stratified k-fold cross-validation. The ablations proved the current evaluation cannot resolve few-point differences; until it can, every other improvement is unmeasurable. Then face cropping at native resolution, which attacks the real bottleneck: information destroyed before it reaches the head.

### Q41. If you had 100x more data, what changes?

Unfreeze more of the backbone (and eventually BN), consider a larger backbone or a dual-stream RGB+frequency design, use identity-disjoint splits, and expect the frequency branch that failed at this scale to start paying off.

### Q42. Would this detect GAN or diffusion deepfakes?

No, and the docs say so. It learned Photoshop composite artefacts (blending, lighting). Generator fingerprints are different statistics; detectors are known to generalise poorly across manipulation families. Cross-dataset evaluation against FaceForensics++ or Celeb-DF is the honest next test, with a large drop expected.

### Q43. Someone wants to use your model to prove an image of a person is fake. What do you say?

Refuse. At 64% accuracy with a 39% false-positive rate on real images, the output must never be treated as evidence about a real person. A positive means 'worth a human look'. That boundary is written into the README and enforced by design through the abstain band.

### Q44. An engineer suggests boosting your accuracy by tuning the threshold on the test set. Response?

That is exactly the failure the project is built to prevent: a threshold tuned on the data you report makes a weak model look strong and destroys the estimate's validity. The threshold comes from validation, ships in the contract, and the test set is opened once.

### Q45. How does Grad-CAM work and what are its limits?

Gradients of the output logit with respect to the last convolutional feature map weight each channel; the weighted sum makes a coarse heatmap of influential regions. Implementation detail: MobileNetV2 is nested, so layer lookup by name fails; the code takes the tensor feeding GlobalAveragePooling2D instead, which works for any backbone. Limits: it shows where the model looked, not whether it was right, and at 7x7 resolution it is coarse.

### Q46. What would production monitoring look like?

Log every prediction's probability and abstain decision; watch the score distribution and abstain rate for drift; sample abstained and high-confidence cases for periodic human labelling; recompute calibration on the labelled stream; alert when ECE or the abstain rate shifts beyond a band.

### Q47. Explain your project to a non-technical manager in three sentences.

We built software that estimates whether a face photo has been Photoshopped. It is right roughly two times out of three, so it is designed as a filter that flags suspicious images for human review rather than a judge, and it deliberately says 'not sure' on the borderline third of cases. Every number we report comes with the statistical context to know how much to trust it.

### Q48. What Keras/TensorFlow version constraints did you hit?

TensorFlow 2.15 has no wheels for Python 3.12+, so the project pins Python 3.11. The TF 2.15 Keras shim also does not expose keras.__version__, handled with a getattr fallback. Small things, but the kind interviewers like hearing you noticed.

### Q49. What is the difference between your smoke test and the unit tests?

The unit tests (pytest, 57) verify component behaviour in isolation with synthetic data in seconds. The smoke command runs the real end-to-end path (one tiny epoch, real wiring, real artifacts) in about 30 seconds; it proves the plumbing, not the accuracy.

### Q50. What single sentence summarises your engineering philosophy in this project?

Every number must be measured, contextualised with a baseline and an interval, and impossible to fake by construction; and the system must be honest about uncertainty, abstaining rather than guessing.

# 10. The cheat card: memorise these

| Fact | Value |
|---|---|
| Dataset | CIPLAB real-and-fake faces, 2,041 images (1,081 real / 960 fake) |
| Fakes are | Expert Photoshop composites (eyes/nose/mouth swaps), NOT GAN output |
| Split | 70/15/15 stratified = 1,429 / 306 / 306; test sealed |
| Model | MobileNetV2 (ImageNet) + GAP + Dropout(0.2) + Dense(1, linear logit) |
| Input | 224 x 224 x 3, raw [0,255]; model rescales to [-1,1] itself |
| Training | Stage 1 frozen @ 1e-4; stage 2 top-40 unfrozen @ 1e-5, BN always frozen |
| Loss | BinaryCrossentropy(from_logits=True); sigmoid(logit) = P(real) |
| Accuracy | 0.6405, CI [0.588, 0.693]; baseline 0.5294; lift +0.111 |
| ROC AUC | 0.6969, CI [0.639, 0.753] |
| Brier / ECE / log loss | 0.2242 / 0.0662 / 0.6461 |
| Threshold | 0.6116, Youden's J on validation, shipped in label_map.json |
| Abstain | Margin 0.10 -> 32% abstain, accuracy on the rest 0.697 |
| Ablations | 4 variants (unfreeze_all, res384, srm, baseline): all within noise; baseline kept |
| Tests | 57, no GPU, no dataset; AST test bans np.random.uniform and r2_score |
| Runtime | Train about 5 min CPU; notebook Run-All about 8 min; Python 3.11 + TF 2.15 |
| One-line ethics | Never evidence about a real person; a positive = 'worth a human look' |

**Final advice.** Do not memorise answers word for word. Understand section 3 (concepts) and section 5 (pipeline) well enough to explain them with a pen and paper, rehearse the 30-second pitch and the two-minute walkthrough aloud, and be ready to say 'the accuracy is modest, here is exactly why, and here is what I would do next': that answer wins more interviews than a big number would.